

Scaling book

How To Scale Your Model

이정우, 박준우

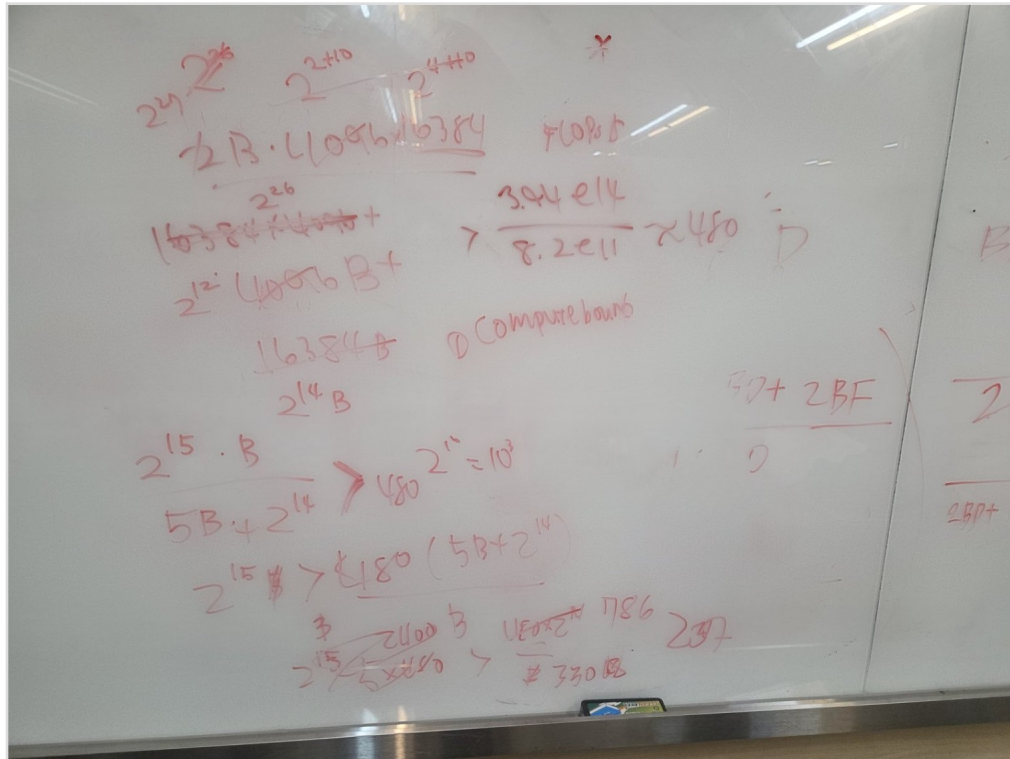
스터디 소개

모델을 효율적으로 학습하고 서빙하는 데 필요한 요소를 계산으로 분석하는 스터디

"벤치마크 점수가 20% 올라도, roofline 효율이 20% 떨어지면 의미가 없다." (교재 서문)

스터디 기록 1 / 1장 Roofline

행렬곱은 언제 compute-bound가 될까

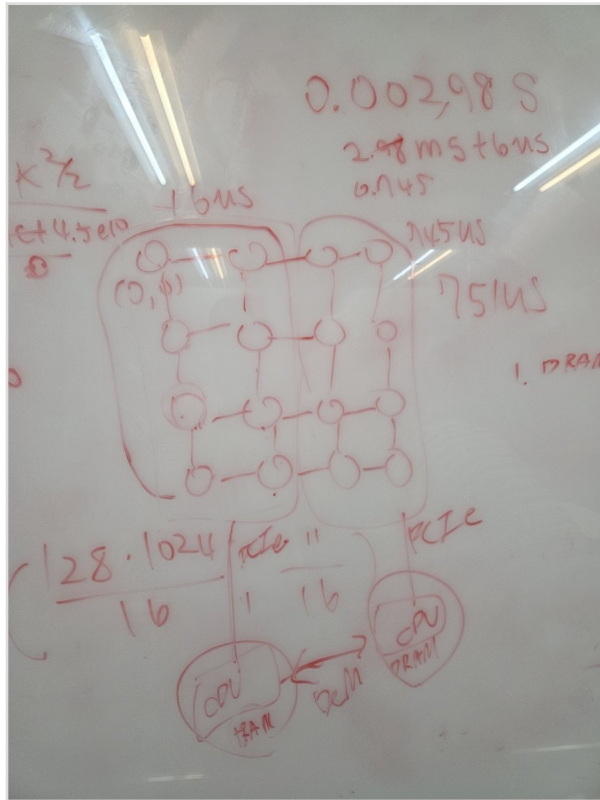


int8 [B, 4096] × [4096, 16384] 행렬곱에서 배치 B는 얼마 이상이어야 할까?

- 연산량: $2 \times B \times 4096 \times 16384$
- 옮기는 데이터: 입력 + 가중치 + 출력
- 칩 기준치: $3.94e14 / 8.2e11 \approx 480$
- 산술 강도 > 480을 풀면 B가 약 240 이상

스터디 기록 2 / 2장 TPU

칩 사이로 데이터를 옮기는 시간



어떤 경로로 옮기느냐에 따라 걸리는 시간이 크게 달라진다.

- 칩과 칩: ICI (옆 칩끼리 직접 연결)
- 칩과 CPU: PCIe
- 호스트와 호스트: DCN
- 전송 시간 $\approx$ 데이터 크기 / 대역폭 + 거쳐 가는 칩 수(hop) $\times$ 지연 시간

스터디 기록 3 / 3장 Sharding

AllGather는 얼마나 걸릴까

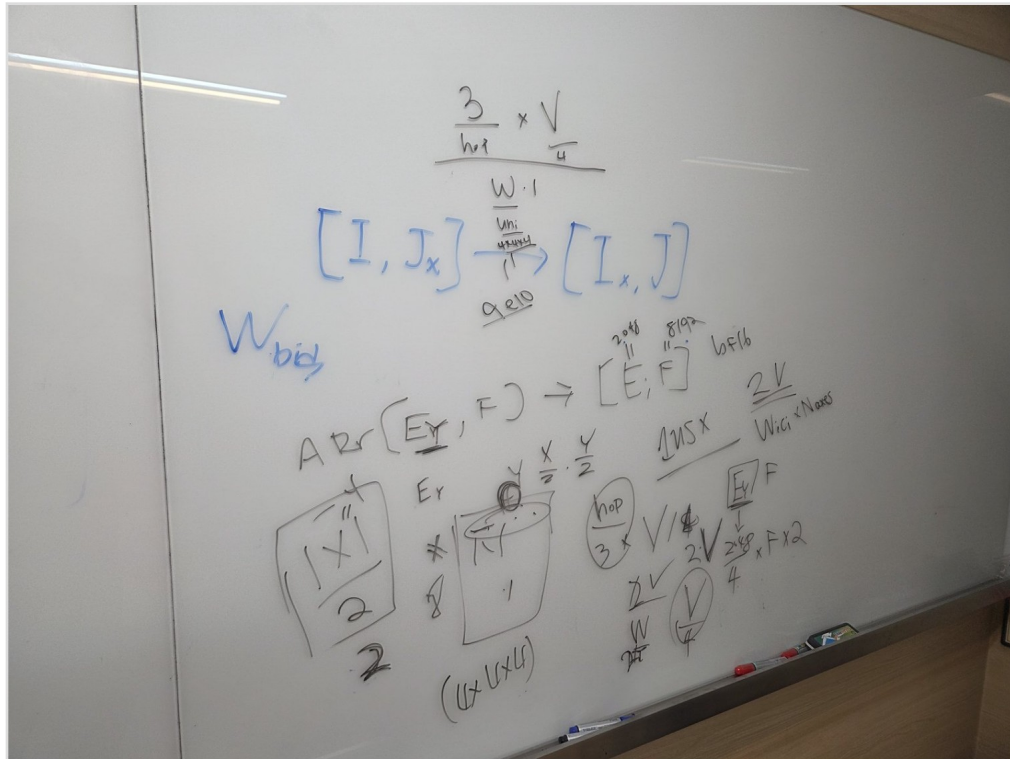
1. All gather T_{comm} $\times 4n$
 $flops = 2BDF \sim$
 $flops = \sim$
 $T_{comm} = \frac{V}{X \cdot W}$
2. $flops_{gn} = 2BDF / x \sim$
 $T_{comm} = \frac{V}{W} \times 2$

Bx 2×128 2×128
 $4 \times 4 \times 4 \times 4$
 Y X $AllGather_X[Bx]$
 $\max \left\{ \frac{T_{min} \times \frac{2 \times 128}{2}}{1 \mu s \times 2}, \frac{V}{W \times \sqrt{N}} \right\}$
 $\frac{2 \times 128}{9 \times 10^4 \times 1}$

통신 시간 = max(지연 시간 하한, 데이터 크기 / 대역폭)
데이터가 작으면 지연 시간이, 크면 대역폭이 시간을 정한다.

스터디 기록 4 / 3장 Sharding

AllToAll과 AllReduce 비용



통신 종류마다 옮기는 데이터 양이 다르다.

- AllToAll $[I, J_X] \rightarrow [I_X, J]$: AllGather의 약 1/4
- AllReduce: ReduceScatter + AllGather라서 약 2배
- 예제: bf16 $[E_Y, F]$ ($E = 2048, F = 8192$)를 $4 \times 4 \times 4$ 칩에서 AllReduce

목차

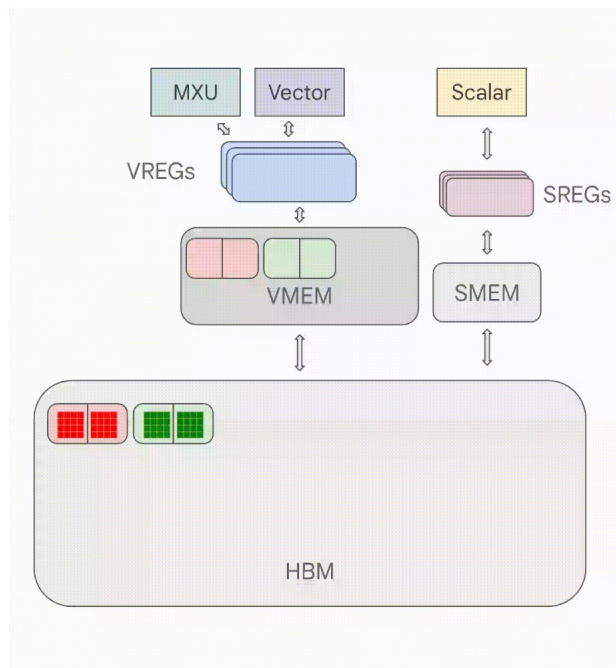
Part	장
Part 1 기초	1 Roofline 분석, 2 TPU 구조, 3 행렬 쪼개서 곱하기
Part 2 Transformer	4 Transformer 계산, 5 학습 병렬화, 6 LLaMA 3 학습, 7 추론, 8 LLaMA 3 서빙
Part 3 실습	9 프로파일링, 10 JAX로 TPU 프로그래밍
Part 4 마무리	11 결론, 12 GPU 관점

Roofline 분석

A Brief Intro to Roofline Analysis

연산 시간은 계산과 데이터 이동 중 더 느린 쪽이 정한다.

- 산술 강도(바이트당 연산 수)가 하드웨어 기준치보다 낮으면 데이터 이동에 막힌다.
- TPU v5e의 기준치는 약 240. 행렬곱은 배치가 240보다 커야 계산 능력을 다 쓴다.



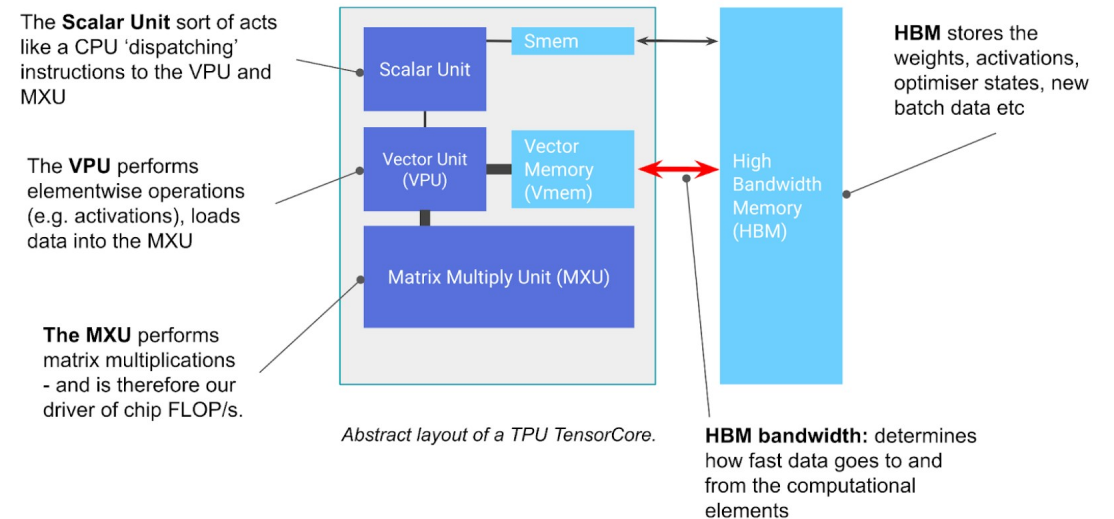
HBM에서 연산 장치로 데이터가 이동하는 과정 (출처: 교재)

TPU 구조

How to Think About TPUs

TPU는 행렬곱 유닛과 메모리 계층으로 이뤄진 단순한 기계다.

- 칩 안: MXU(행렬곱), VPU(그 외 연산), VMEM(작고 빠른 메모리), HBM(큰 메모리)
- 칩 사이: 옆 칩끼리 연결된 토러스 구조(ICI)
- 속도 순서: HBM > ICI > PCIe > DCN



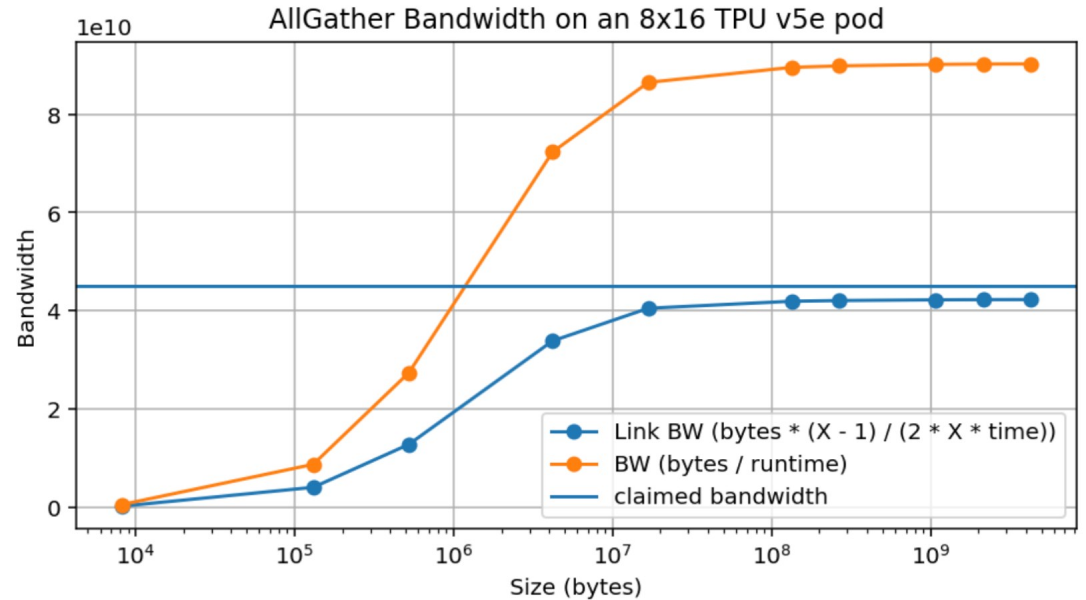
TPU TensorCore 구조 (출처: 교재)

행렬 쪼개서 곱하기

Sharded Matrices and How to Multiply Them

행렬을 어떻게 자르느냐에 따라 필요한 칩 간 통신이 정해진다.

- 통신 4종: AllGather, ReduceScatter, AllReduce, AllToAll
- 대역폭이 병목일 때, 통신 시간은 칩 개수와 거의 상관 없다.



데이터가 작으면 대역폭을 다 못 쓰고, 커지면 한계치에 도달한다 (출처: 교재)

Transformer 계산

All the Transformer Math You Need to Know

Transformer의 파라미터 수, 연산량, 메모리를 직접 계산한다.

- 학습 연산량 $\approx 6 \times \text{파라미터 수} \times \text{토큰 수}$
- 추론할 때 필요한 KV cache 크기 계산

학습 병렬화

How to Parallelize a Transformer for Training

학습을 여러 칩에 나누는 방법과, 각 방법이 통신에 막히는 조건을 분석한다.

- Data, FSDP, Tensor, Pipeline 병렬화
- 여러 방법을 섞으면 칩당 배치가 작아도 계산 능력을 다 쓸 수 있다.

LLaMA 3 학습

Training LLaMA 3 on TPUs

앞의 내용으로 LLaMA 3-70B의 학습 비용을 직접 추정한다.

- 15조 토큰 학습: TPU v5p 8,960개로 약 44일 (활용률 40% 가정)
- 어떤 병렬화 조합을 쓸지 결정

추론

All About Transformer Inference

추론은 prefill과 decode로 나뉘고, 둘의 병목이 다르다.

- prefill(프롬프트 처리): 계산에 막힘. decode(토큰 생성): 메모리에 막힘
- 배치 크기로 지연 시간과 처리량을 맞바꾼다.

LLaMA 3 서빙

Serving LLaMA 3 on TPUs

LLaMA 3-70B를 서빙할 때 필요한 칩 수, 지연 시간, 처리량을 계산한다.

- 긴 문맥에서는 연산보다 KV cache를 읽는 시간이 더 오래 걸린다.
- 지연 시간을 조금 양보하면 토큰당 비용이 크게 줄어든다.

프로파일링

How to Profile TPU Programs

실제로 돌려서 시간이 어디에 쓰이는지 측정한다.

- JAX 프로파일러로 연산별 실행 시간 확인
- roofline으로 예측한 값과 실제 측정값 비교

JAX로 TPU 프로그래밍

Programming TPUs in JAX

여러 칩에 연산을 나누는 코드를 JAX로 직접 작성한다.

- 자동 분할(컴파일러가 통신을 넣어줌)과 `shard_map`(통신을 직접 작성)
- 계산과 통신을 겹쳐서 실행 시간을 줄이는 방법

결론

Conclusions and Further Reading

전체 내용을 정리하고, 더 읽어볼 자료를 소개한다.

- TPU, GPU, JAX 병렬화, Transformer 추론 관련 추가 자료

GPU 관점

How to Think About GPUs

지금까지의 분석을 GPU에 적용한다.

- GPU는 작은 연산 유닛(SM) 수백 개, TPU는 큰 코어 1~2개
- 연결 구조: 노드 안은 NVLink, 노드 사이는 InfiniBand

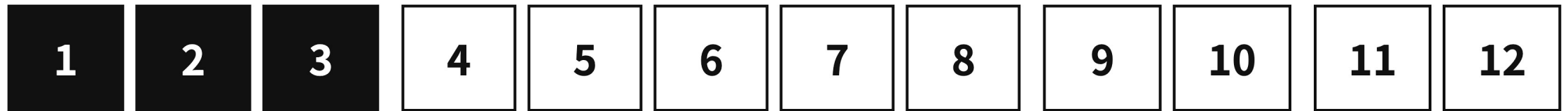
진행 현황

Part 1 기초

Part 2 Transformer

Part 3 실습

Part 4 마무리



검정: 완료 / 흰색: 예정

Part 1(1~3장)까지 완료